work by insisting on punctuation characters, numbers, and so on (widening the pool of potential characters vastly increases the possible permutations). On the other hand, computers, especially modern systems with their focus on parallelism, are brilliant at crunching through data very quickly; a brute force dictionary attack will eventually succeed. What we need to do is find a way to defeat pre-computed hashes; we also need to make every hash in our password store unique even if there are common passwords in there (and in a real user database, there *will* be common passwords).

The Password (Storage) Solution

There is a very simple way to vastly increase the amount of work required to brute force all of your stored passwords and render rainbow tables ineffective. By adding a randomly generated string, known as a *salt*, to every password before performing the hash. The salt does not need to be secret, nor should it be; you're going to need it every time the user authenticates.

```
$password = 'weaksauce';

print sha1($password . random_bytes(8)) . PHP_EOL;
print sha1($password . random_bytes(8));

Output:
2fd158e2411ba5c5da51fe8341ffde81e887dc61
0572ef1520460768a0b9a551091055d1c614c31f
```

Even with two identical passwords, by adding some extra characters before hashing, we have generated two values that will not be easily found in a pre-computed table. Neither can passwords be correlated: the attacker is forced to compute for every password. Every single password in your database could be identical, but the addition of the salt means every hash value is different. Storing the salt alongside the password (e.g., in the same database table) does not present any additional security risk; it serves only to make life more difficult (expensive) for bad actors. Indeed, PHP offers a password hashing function that will return a string that includes the salt, as well as metadata concerning any relevant parameters used in the various algorithms. Using `sha1()` as we've seen so far is not the recommended approach, however, to securing your user's